OF-2 Report :
Simulating flow around a Circular Cylinder

Terry Murray (tnm3843), Nicole Olvera (no4342), Andres Suniaga (aas5778)

03/06/2025

Contents

---

1 Introduction

Flow around a circular cylinder is a classic problem in *fluid mechanics* and is commonly studied in *computational fluid dynamics* (CFD) because of its interesting flow behavior at different Reynolds numbers (Re). At low Re, the flow is steady and symmetric, but as Re increases, the wake behind the cylinder becomes unsteady, first generating recirculation regions and ultimately leading to *vortex shedding* and the formation of a *von Kármán vortex street*. This type of flow is important in many engineering applications where understanding how vortices form and interact with surfaces can help improve designs.
The goal of this project is to **simulate flow past a circular cylinder using OpenFOAM** and analyze how the flow changes at different Reynolds numbers. Specifically, we will:

- **Generate and refine structured meshes** using `blockMesh` to see how grid resolution affects accuracy.
- **Simulate steady and unsteady flow** at $Re = 20$ (where flow remains steady) and $Re = 110$ (where vortex shedding occurs).
- **Analyze velocity, pressure, and forces**, including shear stress and drag.
- **Validate our results** by comparing vortex shedding frequency (Strouhal number) and separation point locations with published data from *Williamson (1988)* and *Park et al. (1998)*.

The simulations will be performed using the **OpenFOAM**, for *incompressible, laminar flow*. The computational setup includes **mesh generation, a no-slip boundary on the cylinder, and downstream data probes**. All variables are nondimensionalized using the cylinder diameter ($D = 1$ m) and freestream velocity ($U = 1$ m/s), which simplifies the Reynolds number to:

$$Re = \frac{UD}{\nu} = \frac{1}{\nu}. \tag{1}$$

In this report, we will go through the **mesh setup, simulation process, and post-processing results using ParaView** to understand how the flow behaves at different Reynolds numbers. By refining our meshes and analyzing key flow parameters, we aim to gain a better understanding of *how OpenFOAM handles external flow simulations and how mesh quality affects the results.*

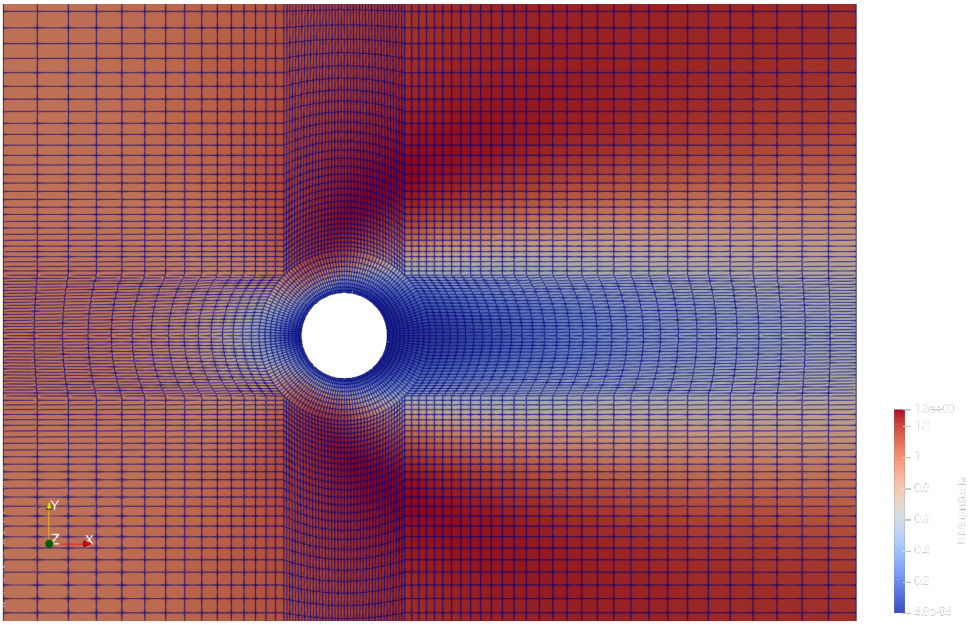


Figure 1: Visualization of the computational domain and mesh in ParaView.

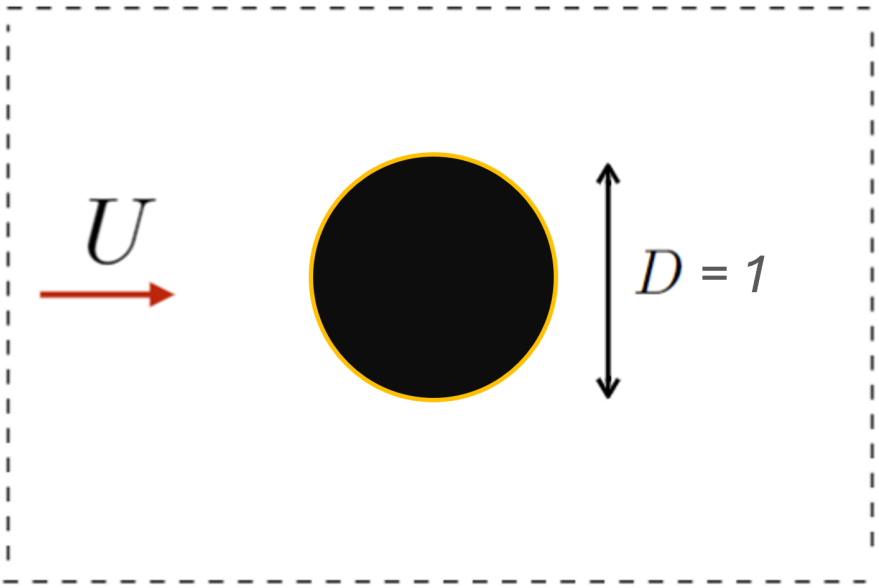


Figure 2: Structured mesh used for the simulation.

To illustrate the computational setup, Figure 1 shows the domain and mesh visualized in ParaView, and Figure 2 provides a detailed circular cylindar used in the study.

2 Assembly of preliminary meshes

2.1 Schematic of Mesh Generation

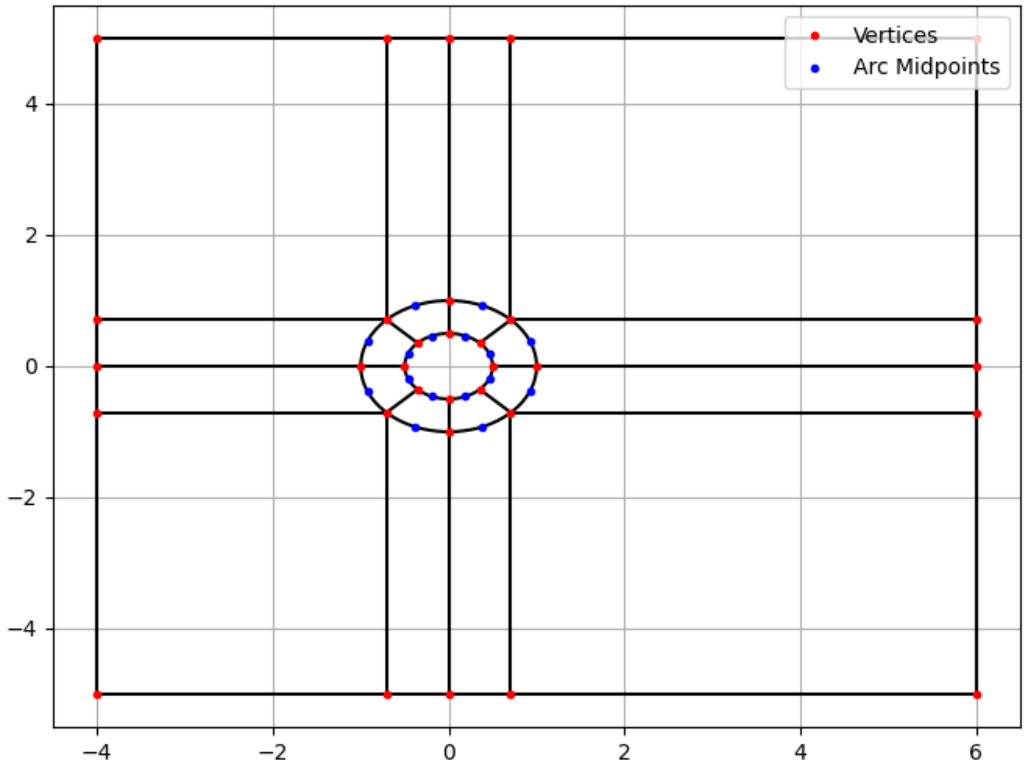


Figure 3: Generating the block mesh in $\mathbb{R}^2$

The block mesh is characterized by the fore length L_f, wake length L_w, half-height H, outer circle radius R, and the diameter of the circular cylinder D. These parameters can be changed within the code (found in the Appendix) to output new vertices, edges, arc points, and faces. There are 32 vertices for simplicity to match with the example given in lecture.

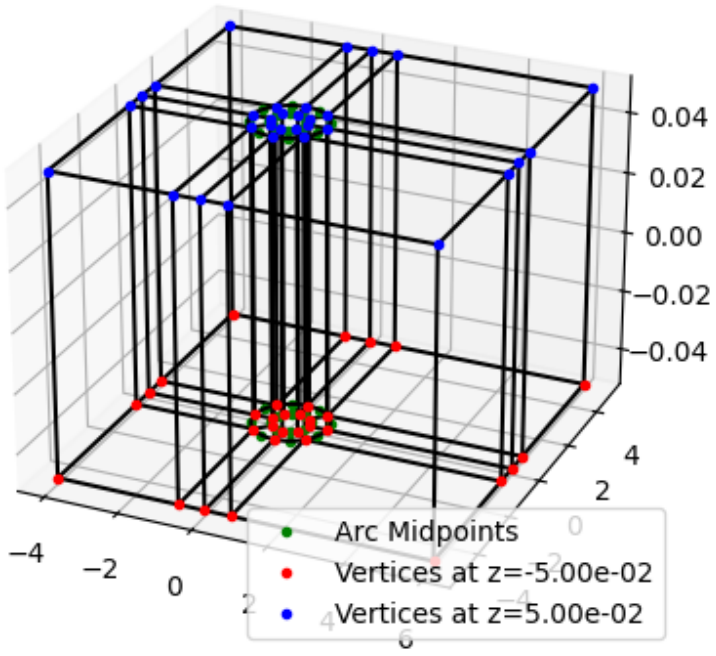


Figure 4: Visualization in $\mathbb{R}^3$

Since OpenFOAM is inherently a 3-dimensional solver, we generate our blocks in $\mathbb{R}^3$ within a very small z margin of -0.05 to 0.05. This is the trick to keep it, for the most part, 2-dimensional. NOTE: Code included in the Appendix to reproduce meshes.

2.2 Mesh files

Attached with report submission. Also in the teams Github linked in the Appendix.

2.3 Visualization

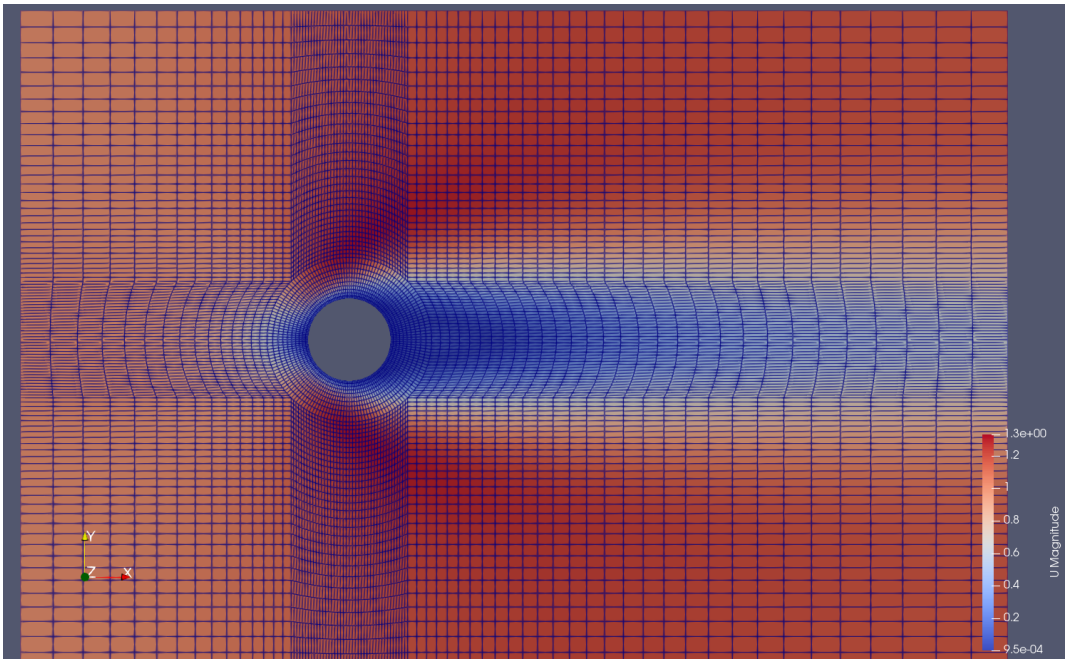


Figure 5: Mesh A in *ParaView*

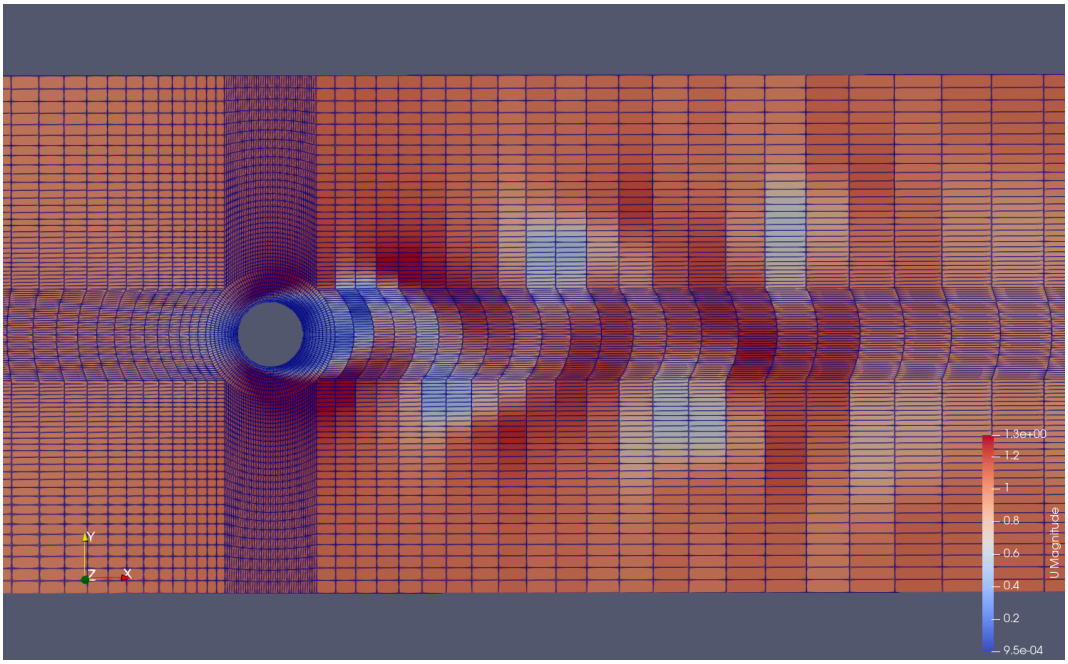
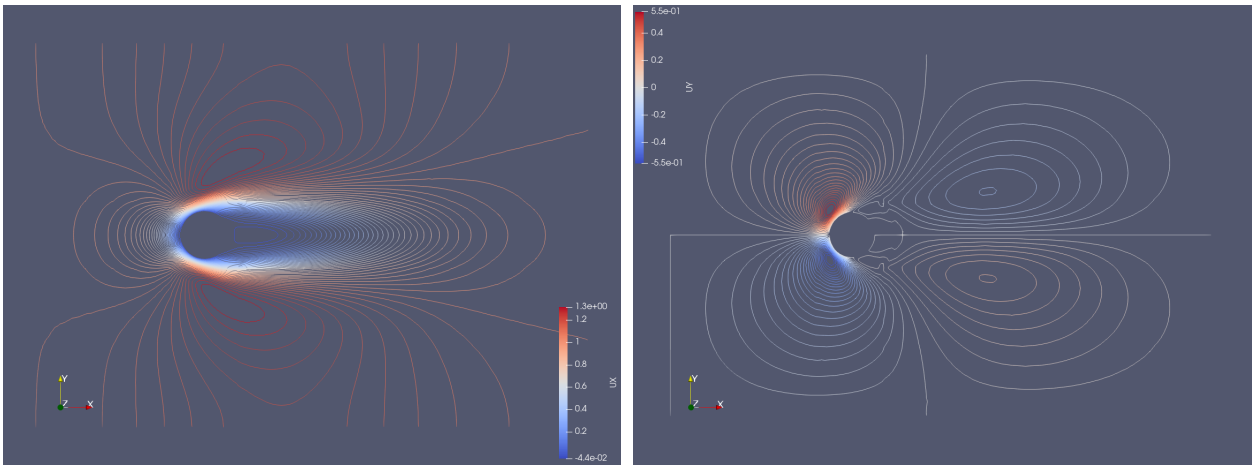


Figure 6: Mesh B in *ParaView*

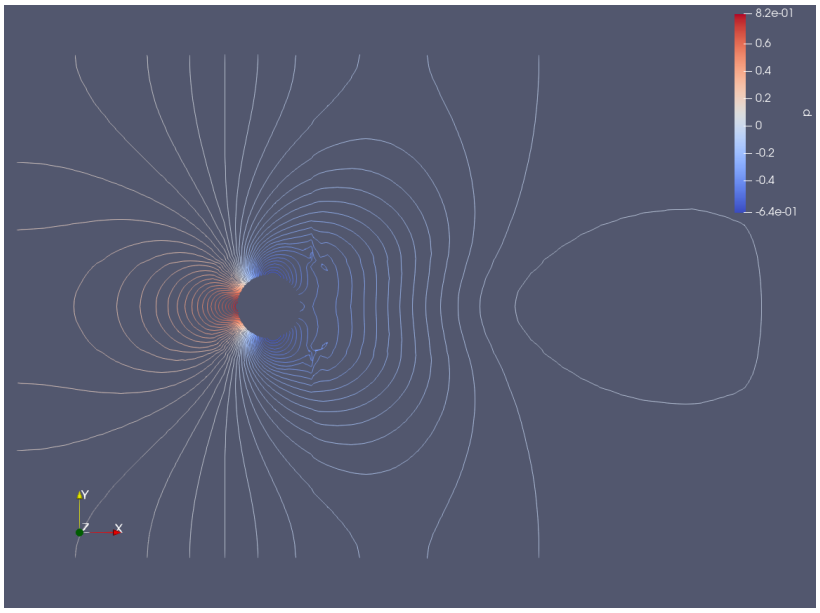
A longer wake region, L_w, is chosen for Mesh B to observe the vortex shedding.

3 Solution for Re=20 and 110

3.1 Re=20



(a) u/U (b) v/U



(c) $p/\rho U^2$

Figure 7: Velocity and pressure distributions for steady solution

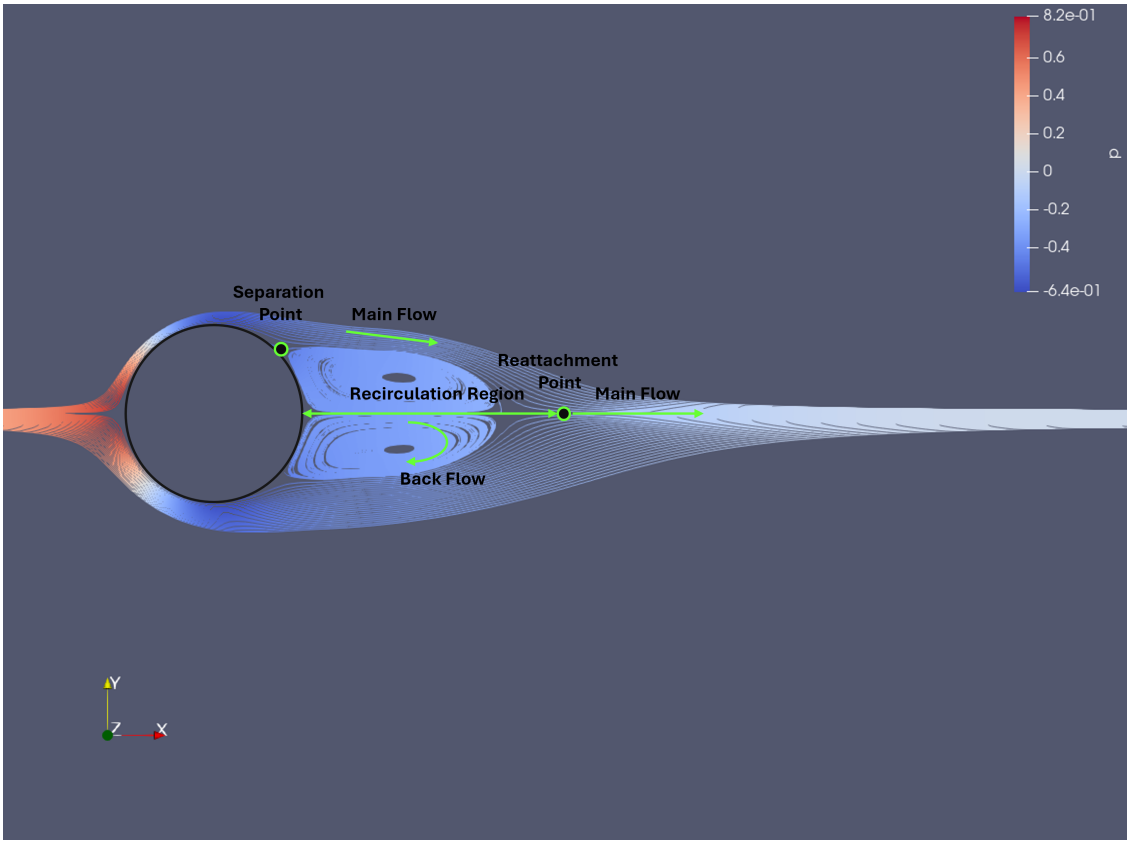
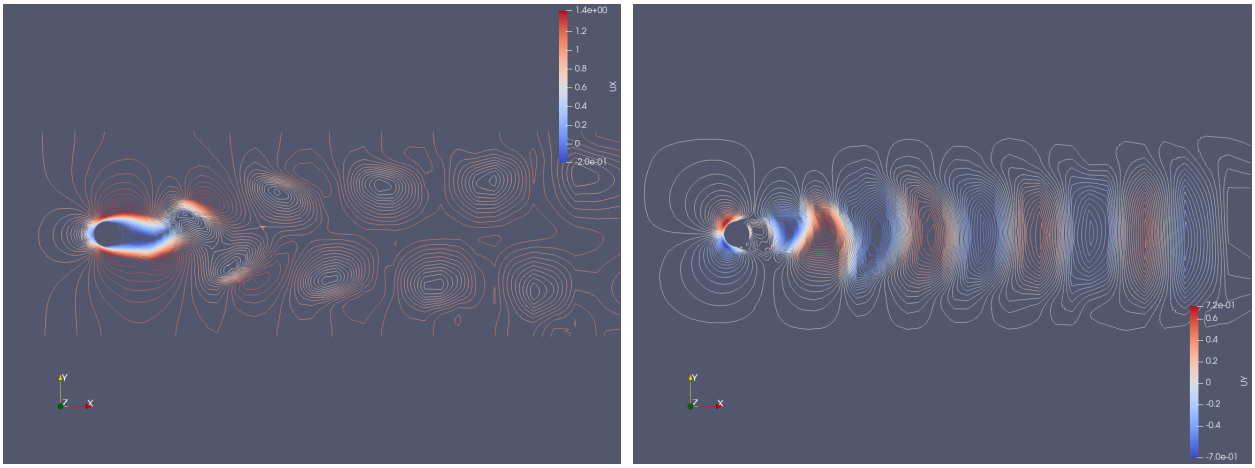
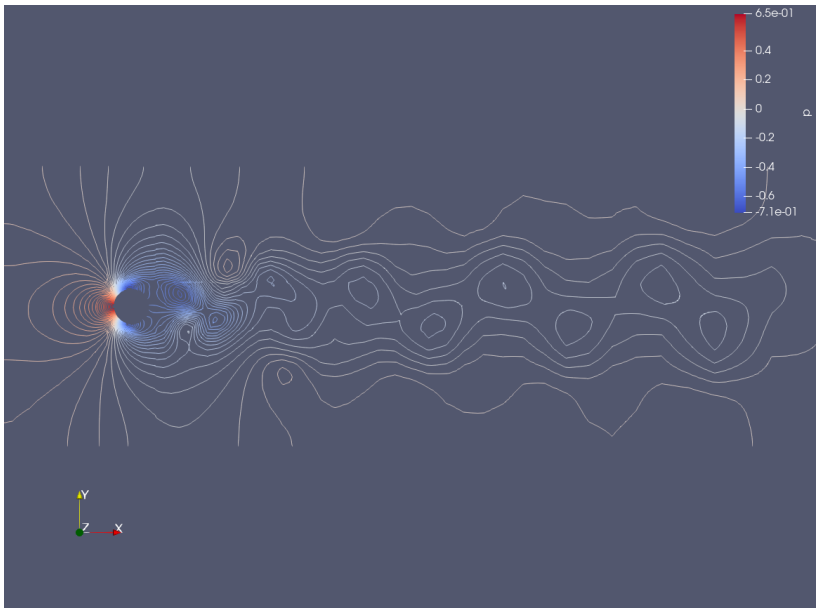


Figure 8: Visulaizing streamlines for steady solution

3.2 Re=110



(a) u/U (b) v/U



(c) $p/\rho U^2$

Figure 9: Velocity and pressure distributions for the unsteady solution

4 Improving the Mesh

5 Unsteady Flow: Strouhal Number of the vortex shedding

6 Steady Flow: velocity in the boundary layer, separation length, and rate of strain at the cylinder's wall

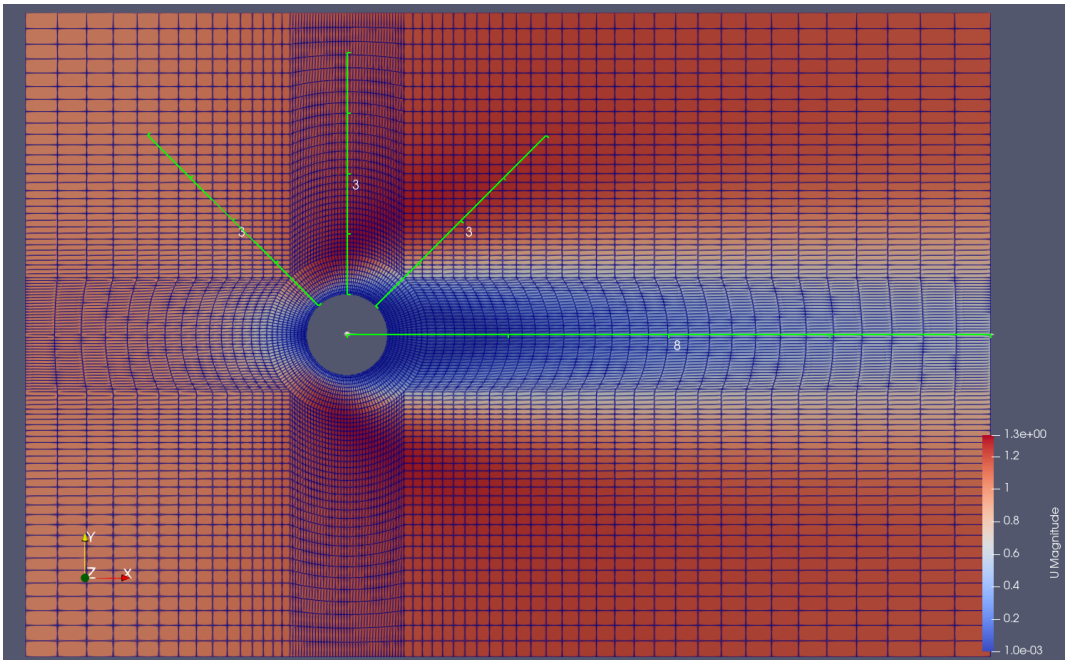


Figure 10: Lines at angles $\theta = \pi/4,\ \pi/2,\ 3\pi/4$ and horizontal line for finding L/D

6.1 Radial and Tangential velocity

We will extract u and v components of velocity along lines at angles $\theta = \{\pi/4,\ \pi/2,\ 3\pi/4\}$.

6.2 Rate of strain

6.3 Length of recirculation region

For finding the length of the recirculation region we will find the point at which velocity reaches 0 and find the difference in that x-position to point at which the recirculation region begins, i.e. $x = 0.5$.

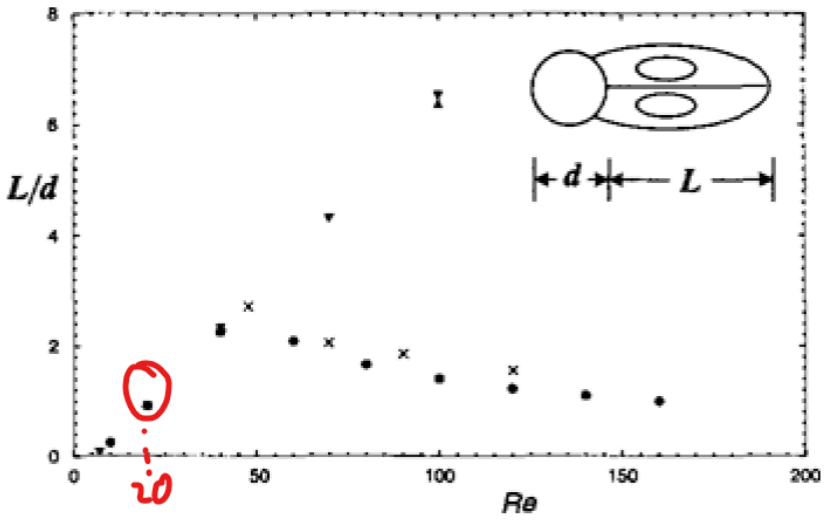


Figure 11: L/d vs Re in *Park* paper,$L/d \approx 1$ at $Re = 20$

7 Drag Coefficient for the circular cylinder

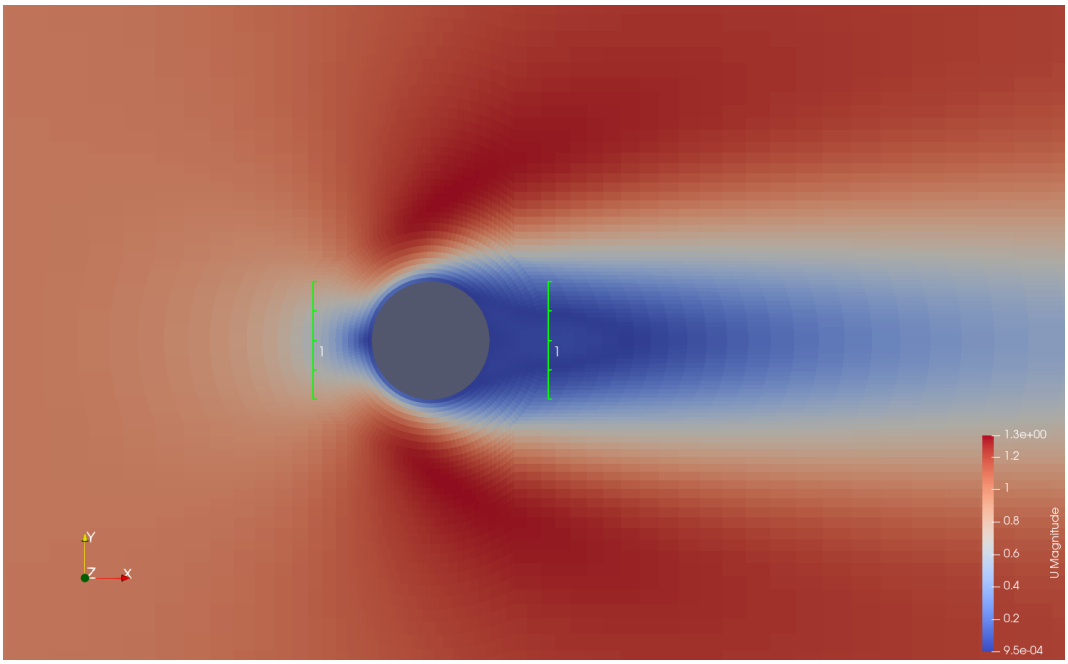


Figure 12: Image of lines used for post processing

Appendix

A Code

PDF of code starts on next page.

Additionally, team repository: `https://github.com/andressuniaga/CFD-SP25-11`

Generate Block Mesh Dict

```
import math

# ======== Generates the vertices ========
def circle_points(radius, z):
      """
      Returns a list of 8 coordinates around a circle of radius R (defines
8-15)
      """
      angles = [0, 45, 90, 135, 180, 225, 270, 315]
      coordinates = []
      for i in angles:
      theta = math.radians(i)   # measured in radians so convert
      x = radius * math.cos(theta)
      y = radius * math.sin(theta)
      coordinates.append((x, y, z))
      return coordinates

def domain_boundary_points(Lf, Lw, H, R, z):
      """
      Returns 16 coordinates for the outer rectangle
      """
      r_diag = R / math.sqrt(2)
      return [
      (Lw,  0.0,   z),   # 16
      ( Lw, r_diag,z),   # 17
      ( Lw, H,    z),   # 18
      ( r_diag,H,       z),   # 19
      ( 0.0,   H,       z),   # 20
      (-r_diag,H,       z),   # 21
      (-Lf, H,    z),   # 22
      (-Lf, r_diag,z),   # 23
      (-Lf, 0.0,   z),   # 24
      (-Lf,   -r_diag,z),   # 25
      (-Lf,   -H,       z),   # 26
      (-r_diag,-H,      z),   # 27
      ( 0.0,   -H,      z),   # 28
      ( r_diag,-H,      z),   # 29
      ( Lw, -H,   z),   # 30
      ( Lw,   -r_diag,z),   # 31
```

```
      ]

def create_vertices(Lf, Lw, H, R_inner, R, z):
      """
      Build all 64 vertex coordinates in correct order
      """
      # defining 0-31 points
      inner_circle = circle_points(R_inner, -z)   # could also put this in
+z
      outer_circle = circle_points(R, -z)
      rectangle = domain_boundary_points(Lf, Lw, H, R, -z)

      # replicate but now at +z
      inner_circle_pos = circle_points(R_inner, z)
      outer_circle_pos = circle_points(R, z)
      rectangle_pos = domain_boundary_points(Lf, Lw, H, R, z)

      vertices = (inner_circle + outer_circle + rectangle +
inner_circle_pos + outer_circle_pos + rectangle_pos)

      return vertices

def write_vertices_block(vertices):
      lines = []
      lines.append("vertices\n(")
      for i, (x, y, z) in enumerate(vertices):
      lines.append(f"   ( {x:.8e} {y:.8e} {z:.8e} ) // {i}")
      lines.append(");")
      return "\n".join(lines)


# ======== Blocks ========
def define_blocks():
      # should stay the same, we will still have the same amount of blocks
      blocks = []

      blocks.append({
      "verts": (0, 8, 9, 1, 32, 40, 41, 33),
      "cells": (10, 20, 1),
      "grading": (2.0, 1.0, 1.0)
      })
      blocks.append({
      "verts": (1, 9, 10, 2, 33, 41, 42, 34),
```

```
    "cells": (10, 20, 1),
    "grading": (2.0, 1.0, 1.0)
    })
    blocks.append({
    "verts": (2, 10, 11, 3, 34, 42, 43, 35),
    "cells": (10, 20, 1),
    "grading": (2.0, 1.0, 1.0)
    })
    blocks.append({
    "verts": (3, 11, 12, 4, 35, 43, 44, 36),
    "cells": (10, 20, 1),
    "grading": (2.0, 1.0, 1.0)
    })
    blocks.append({
    "verts": (4, 12, 13, 5, 36, 44, 45, 37),
    "cells": (10, 20, 1),
    "grading": (2.0, 1.0, 1.0)
    })
    blocks.append({
    "verts": (5, 13, 14, 6, 37, 45, 46, 38),
    "cells": (10, 20, 1),
    "grading": (2.0, 1.0, 1.0)
    })
    blocks.append({
    "verts": (6, 14, 15, 7, 38, 46, 47, 39),
    "cells": (10, 20, 1),
    "grading": (2.0, 1.0, 1.0)
    })
    blocks.append({
    "verts": (7, 15, 8, 0, 39, 47, 40, 32),
    "cells": (10, 20, 1),
    "grading": (2.0, 1.0, 1.0)
    })
    blocks.append({
    "verts": (8, 16, 17, 9, 40, 48, 49, 41),
    "cells": (30, 20, 1),
    "grading": (4.0, 1.0, 1.0)
    })
    blocks.append({
    "verts": (9, 17, 18, 19, 41, 49, 50, 51),
    "cells": (30, 30, 1),
    "grading": (4.0, 4.0, 1.0)
    })
```

```
    blocks.append({
    "verts": (10, 9, 19, 20, 42, 41, 51, 52),
    "cells": (20, 30, 1),
    "grading": (1.0, 4.0, 1.0)
    })
    blocks.append({
    "verts": (11, 10, 20, 21, 43, 42, 52, 53),
    "cells": (20, 30, 1),
    "grading": (1.0, 4.0, 1.0)
    })
    blocks.append({
    "verts": (23, 11, 21, 22, 55, 43, 53, 54),
    "cells": (15, 30, 1),
    "grading": (0.25, 4.0, 1.0)
    })
    blocks.append({
    "verts": (24, 12, 11, 23, 56, 44, 43, 55),
    "cells": (15, 20, 1),
    "grading": (0.25, 1.0, 1.0)
    })
    blocks.append({
    "verts": (25, 13, 12, 24, 57, 45, 44, 56),
    "cells": (15, 20, 1),
    "grading": (0.25, 1.0, 1.0)
    })
    blocks.append({
    "verts": (26, 27, 13, 25, 58, 59, 45, 57),
    "cells": (15, 30, 1),
    "grading": (0.25, 0.25, 1.0)
    })
    blocks.append({
    "verts": (27, 28, 14, 13, 59, 60, 46, 45),
    "cells": (20, 30, 1),
    "grading": (1.0, 0.25, 1.0)
    })
    blocks.append({
    "verts": (28, 29, 15, 14, 60, 61, 47, 46),
    "cells": (20, 30, 1),
    "grading": (1.0, 0.25, 1.0)
    })
    blocks.append({
    "verts": (29, 30, 31, 15, 61, 62, 63, 47),
    "cells": (30, 30, 1),
```

```
        "grading": (4.0, 0.25, 1.0)
        })
        blocks.append({
        "verts": (15, 31, 16, 8, 47, 63, 48, 40),
        "cells": (30, 20, 1),
        "grading": (4.0, 1.0, 1.0)
        })

        return blocks

def write_blocks_section(block_list):
        lines = []
        lines.append("blocks")
        lines.append("(")
        for i, block in enumerate(block_list):
        v = block["verts"]
        c = block["cells"]
        g = block["grading"]
        lines.append(f"   hex ({v[0]} {v[1]} {v[2]} {v[3]} {v[4]} {v[5]}
{v[6]} {v[7]}) " +
                     f"({c[0]} {c[1]} {c[2]}) " +
                     f"simpleGrading ({g[0]} {g[1]} {g[2]}) // block {i}")
        lines.append(");")
        return "\n".join(lines)


# ======== Generates edges ========
def compute_arc_point(v1, v2):
        x1, y1, z1 = v1
        x2, y2, z2 = v2

        # compute angles of each vertex
        theta1 = math.atan2(y1, x1)
        theta2 = math.atan2(y2, x2)

        avg_cos = math.cos(theta1) + math.cos(theta2)
        avg_sin = math.sin(theta1) + math.sin(theta2)
        avg_theta = math.atan2(avg_sin, avg_cos)

        r1 = math.hypot(x1, y1)
        r2 = math.hypot(x2, y2)
        r = 0.5 * (r1 + r2)
        return (r * math.cos(avg_theta), r * math.sin(avg_theta), z1)
```

```
def write_edges_section(vertices):
      lines = []
      lines.append("edges")
      lines.append("(")

      # negative z inner circle (vertices 0-7)
      for i in range(8):
      v1 = vertices[i]
      v2 = vertices[(i + 1) % 8]
      arc_pt = compute_arc_point(v1, v2)
      lines.append(f"   arc {i} {(i+1)%8} ( {arc_pt[0]:.5e} {arc_pt[1]:.5e}
{arc_pt[2]:.5e} )")

      # negative z outer circle
      for i in range(8, 16):
      v1 = vertices[i]
      v2 = vertices[8 + ((i - 8 + 1) % 8)]
      arc_pt = compute_arc_point(v1, v2)
      lines.append(f"   arc {i} {8 + ((i-8+1)%8)} ( {arc_pt[0]:.5e}
{arc_pt[1]:.5e} {arc_pt[2]:.5e} )")

      # positive z inner circle
      for i in range(32, 40):
      v1 = vertices[i]
      v2 = vertices[32 + ((i - 32 + 1) % 8)]
      arc_pt = compute_arc_point(v1, v2)
      lines.append(f"   arc {i} {32 + ((i-32+1)%8)} ( {arc_pt[0]:.5e}
{arc_pt[1]:.5e} {arc_pt[2]:.5e} )")

      # positive z outer circle
      for i in range(40, 48):
      v1 = vertices[i]
      v2 = vertices[40 + ((i - 40 + 1) % 8)]
      arc_pt = compute_arc_point(v1, v2)
      lines.append(f"   arc {i} {40 + ((i-40+1)%8)} ( {arc_pt[0]:.5e}
{arc_pt[1]:.5e} {arc_pt[2]:.5e} )")

      lines.append(");")
      return "\n".join(lines)


# ======== Boundaries ========
```

```
def write_boundary_section():
      boundary = """boundary
(

  inlet
  {
      type patch;
      faces
      (
      (22 54 55 23)
      (23 55 56 24)
      (24 56 57 25)
      (25 57 58 26)
      );
  }

  outlet
  {
      type patch;
      faces
      (
      (18 50 49 17)
      (17 49 48 16)
      (16 48 63 31)
      (31 63 62 30)
      );
  }

  cylinder
  {
      type wall;
      faces
      (
      (0 32 33 1)
      (1 33 34 2)
      (2 34 35 3)
      (3 35 36 4)
      (4 36 37 5)
      (5 37 38 6)
      (6 38 39 7)
      (7 39 32 0)
      );
  }
```

```
    top
    {
        type symmetryPlane;
        faces
        (
        (22 54 53 21)
        (21 53 52 20)
        (20 52 51 19)
        (19 51 50 18)
        );
    }

    bottom
    {
        type symmetryPlane;
        faces
        (
        (26 58 59 27)
        (27 59 60 28)
        (28 60 61 29)
        (29 61 62 30)
        );
    }

);
"""
        return boundary

def main():
        # define parameters
        Lf = 4.0
        Lw = 20.0
        H = 4.0
        R_inner = 0.5
        R = 1.0
        z = 0.05
        convertToMeters = 1.0

        vertices = create_vertices(Lf, Lw, H, R_inner, R, z)
        vertices_text = write_vertices_block(vertices)
        block_data_list = define_blocks()
        blocks_text = write_blocks_section(block_data_list)
```

```
    edges_text = write_edges_section(vertices)
    boundary_text = write_boundary_section()

    bmd = []
    bmd.append("FoamFile")
    bmd.append("{")
    bmd.append("      version  2.0;")
    bmd.append("      format   ascii;")
    bmd.append("      class dictionary;")
    bmd.append("      object   blockMeshDict;")
    bmd.append("}")
    bmd.append("")
    bmd.append("convertToMeters {};".format(convertToMeters))
    bmd.append("")
    bmd.append(vertices_text)
    bmd.append("\n")
    bmd.append(blocks_text)
    bmd.append("\n")
    bmd.append(edges_text)
    bmd.append(boundary_text)

    complete_text = "\n".join(bmd)

    # write to a file here
    with open("blockMeshDict_new", "w") as f:
    f.write(complete_text)
    print("blockMeshDict file has been generated")

if __name__ == "__main__":
    main()
```

Block Mesh Schematic Generation

```
# Script for OF2 : blockmeshvertices

# outputs x-y vertices


#------------------------------#

import numpy as np
import matplotlib.pyplot as plt

#------------------------------#

def drawcircle(R):
      theta = np.linspace(0,2*np.pi,1000)
      circ=np.zeros([2,len(theta)])
      for i in range(len(theta)):
      circ[0,i] = R*np.cos(theta[i])
      circ[1,i] = R*np.sin(theta[i])
      return circ

def arcvertices(D,R,N=32):
      '''
      Inputs:
      N = Number of vertices
      D = Diameter of circular cylinder
      R = Distance between circular cylinder and outer circle
      '''
      if N%16!=0 or N<32:
      print("Ensure N is divisible by 16 and at least 32.")
      return None
      else:
      num = N//4
      theta = np.linspace(0,2*np.pi,num+1)
      pinner = np.zeros([2,num])
      pouter = np.zeros([2,num])
      for i in range(num):
            pinner[0,i] = (D/2)*np.cos(theta[i])
            pinner[1,i] = (D/2)*np.sin(theta[i])

            pouter[0,i] = (R)*np.cos(theta[i])
```

```
            pouter[1,i] = (R)*np.sin(theta[i])

      return pinner, pouter

def walls(Lf,Lw,H,outer, N=32):
      '''
      Inputs:
      N = Number of vertices
      Lf = Length at the fore of the circular cylinder
      Lw = Length at the wake of the circular cylinder
      H = Half height of inlet/outlet
      outer = The outer circle's vertices
      D = Diameter of circular cylinder
      R = Distance between circular cylinder and outer circle
      '''
      if N%16!=0 or N<32:
      print("Ensure N is divisible by 16 and at least 32.")
      return None
      else:
      corners = np.zeros([2,4]) # always four corners to a square
      for i in range(4):
            a = -Lf
            b = H
            if i%2==0:
                  a = Lw
            if i > 1:
                  b = -H
            corners[0,i] = a
            corners[1,i] = b

      inlet = np.zeros([2,N//8])
      top = np.zeros([2,N//8])
      bottom = np.zeros([2,N//8])
      outlet = np.zeros([2,N//8])

      for i in range(N//8):
            top[1,i] = H
            bottom[1,i] = -H
            inlet[0,i] = -Lf
            outlet[0,i] = Lw

            if -outer[0,0]<outer[0,i]<outer[0,0]:
                  top[0,i] = outer[0,i]
```

```
                bottom[0,i] = outer[0,i]

            if outer[1,i] < max(outer[1]):
                inlet[1,i] = outer[1,i]
                outlet[1,i] = outer[1,i]

        for i in range(1,N//8//2):
                inlet[1,-i] = -inlet[1,i]
                outlet[1,-i] = -outlet[1,i]

        # putting in corner points
        outlet[0,-N//16] = corners[0,0]
        outlet[1,-N//16] = corners[1,0]

        top[0,-N//16] = corners[0,1]
        top[1,-N//16] = corners[1,1]

        bottom[0,-N//16] = -corners[0,2] # negated for programming purposes
        bottom[1,-N//16] = corners[1,2]

        inlet[0,-N//16] = corners[0,3]
        inlet[1,-N//16] = corners[1,3]

        topnew = np.zeros([2,N//8])
        bottomnew = np.zeros([2,N//8])
        inletnew = np.zeros([2,N//8])

        a = (N//8)//2 - 1 #index to start and rearrange points
        for i in range(N//8):
            topnew[0,i] = top[0,a-i]
            topnew[1,i] = top[1,a-i]
            bottomnew[0,i] = -bottom[0,a-i]
            bottomnew[1,i] = bottom[1,a-i]
            inletnew[0,i] = inlet[0,a-i]
            inletnew[1,i] = inlet[1,a-i]

        top = topnew
        bottom = bottomnew
        inlet = inletnew

        return top,bottom,inlet,outlet

def blockmesh_2D(inner,outer,inlet,top,outlet,bottom,N=32):
```

```
    if N%16!=0 or N<32:
    print("Ensure N is divisible by 16 and at least 32.")
    return None
    else:
    # Coding up the 2D Block Mesh vertices in order
    # refer to helpful\OF2\10.OF-tutorial-2-Part-2.2019.03.05.pdf
    points = np.zeros([2,N])

    for i in range(N//2): # circular points
          if i < N//4:
                points[0,i] = inner[0,i]
                points[1,i] = inner[1,i]
          else:
                points[0,i] = outer[0,i-N//4]
                points[1,i] = outer[1,i-N//4]

    together_x = []
    together_y = []

    # ugly section of code but works
    for i in range(N//8):
          if outlet[1,i] >= 0:
                together_x.append(outlet[0,i])
                together_y.append(outlet[1,i])
    for i in range(N//8):
          together_x.append(top[0,i])
          together_y.append(top[1,i])
    for i in range(N//8):
          together_x.append(inlet[0,i])
          together_y.append(inlet[1,i])
    for i in range(N//8):
          together_x.append(bottom[0,i])
          together_y.append(bottom[1,i])

    for i in range(len(together_x)):
          points[0,i+N//2] = together_x[i]
          points[1,i+N//2] = together_y[i]

    for i in range(1,N//8//2):
          points[0,-i] = outlet[0,-i]
          points[1,-i] = outlet[1,-i]

    return points
```

```
def arcmidpoints(z,points,D,R,N=32):
      midpoints = np.zeros([3,N//2])
      midpoints[2,:] = z

      for i in range(N//2-1):
      if i<N//4-1:
            theta_x = np.acos((points[0,i+1]*2/D)) +
np.acos((points[0,i]*2/D))
            theta_y = np.asin((points[1,i+1]*2/D)) +
np.asin((points[1,i]*2/D))
            midpoints[0,i] = D/2*np.cos(theta_x/2)
            midpoints[1,i] = D/2*np.sin(theta_y/2)
      elif i == N//4-1:
            theta_x = np.acos((points[0,0]*2/D)) +
np.acos((points[0,i]*2/D))
            theta_y = np.asin((points[1,0]*2/D)) +
np.asin((points[1,i]*2/D))
            midpoints[0,i] = D/2*np.cos(theta_x/2)
            midpoints[1,i] = D/2*np.sin(theta_y/2)
      elif N//4-1<i<N//2-1:
            theta_x = np.acos((points[0,i+1]/R)) + np.acos((points[0,i]/R))
            theta_y = np.asin((points[1,i+1]/R)) + np.asin((points[1,i]/R))
            midpoints[0,i] = R*np.cos(theta_x/2)
            midpoints[1,i] = R*np.sin(theta_y/2)

      theta_x = np.acos((points[0,N//4]/R)) + np.acos((points[0,N//2-1]/R))
      theta_y = np.asin((points[1,N//4]/R)) + np.asin((points[1,N//2-1]/R))
      midpoints[0,-1] = R*np.cos(theta_x/2)
      midpoints[1,-1] = R*np.sin(theta_y/2)


      return midpoints



if __name__ == "__main__":
      # Initial Parameters
      N = 32 # number of points (multiples of 16)

      D = 1 # diameter of cylinder

      Lf = 4
```

```
Lw = 6
R = 1
H = 5

cylinder,outer = arcvertices(D,R,N)
top,bottom,inlet,outlet = walls(Lf,Lw,H,outer,N)

# 2D PLOT
plt.figure(figsize=(8,6))

print("top:\n", top,'\n',"bottom:\n",bottom,'\n'
      "inlet:\n", inlet,'\n',"outlet:\n",outlet,'\n')

vertices = blockmesh_2D(cylinder,outer,inlet,top,outlet,bottom,N)
z=0
arcP = arcmidpoints(z,vertices,D,R)

print(arcP)

# for arcs
cylinder = drawcircle(D/2)
boundarycircle = drawcircle(R)


# for box
verts = np.zeros([2,N//2+1])
for i in range(N//2):
verts[0,i] = vertices[0,i+N//2]
verts[1,i] = vertices[1,i+N//2]
verts[0,-1] = vertices[0,16]
verts[1,-1] = vertices[1,16]

# inside edges
edgesin = np.zeros([2,N//2])
j = 1
for i in range(N//2):
if i%2==0:
      edgesin[0,i] = vertices[0,i//2]
      edgesin[1,i] = vertices[1,i//2]
else:
      edgesin[0,i] = vertices[0,i+N//4-j]
      edgesin[1,i] = vertices[1,i+N//4-j]
      j += 1
```

```
# outside edges
j = 1
edgesout = np.zeros([2,N//2])
for i in range(N//2):
if i%2==0:
      edgesout[0,i] = vertices[0,(i//2+N//4)]
      edgesout[1,i] = vertices[1,(i//2+N//4)]
else:
      if j%3==0:
            j = 1
            edgesout[0,i] = vertices[0,i+N//2-j]
            edgesout[1,i] = vertices[1,i+N//2-j]
      else:
            edgesout[0,i] = vertices[0,i+N//2-j]
            edgesout[1,i] = vertices[1,i+N//2-j]
      j+=1

# remaining outside edges
x_remain = []
y_remain = []
p = 2
for i in range(N//8):
x_remain.append(N//4+1+i*2)
y_remain.append(N//2+3+2*i*p)

edgesout_remain = np.zeros([2,N//4])
j = 1
for i in range(N//4):
if i%2==0:
      edgesout_remain[0,i] = vertices[0,x_remain[i//2]]
      edgesout_remain[1,i] = vertices[1,x_remain[i//2]]
else:
      edgesout_remain[0,i] = vertices[0,y_remain[i-j]]
      edgesout_remain[1,i] = vertices[1,y_remain[i-j]]
      j+=1

for i in range(N//2):
plt.plot(edgesin[0,0+2*i:2+2*i],edgesin[1,0+2*i:2+2*i],'k')

for i in range(N//2):
plt.plot(edgesout[0,0+2*i:2+2*i],edgesout[1,0+2*i:2+2*i],'k')
```

```
      for i in range(N//4):

plt.plot(edgesout_remain[0,2*i:2+2*i],edgesout_remain[1,2*i:2+2*i],'k')



plt.plot(cylinder[0],cylinder[1],'k',boundarycircle[0],boundarycircle[1],'k
')
      plt.plot(verts[0],verts[1],'k')
      plt.plot(vertices[0],vertices[1],'r.',label="Vertices")
      plt.plot(arcP[0,:],arcP[1,:],'b.',label="Arc Midpoints")
      plt.grid()
      plt.legend(fontsize=10)

      # 3D PLOT
      plt.figure()
      ax = plt.axes(projection = '3d')
      x = vertices[0]
      y = vertices[1]
      z1 = -5e-02
      z2 = 5e-02

      arcpoints1 = arcmidpoints(z1,vertices,D,R)
      arcpoints2 = arcmidpoints(z2,vertices,D,R)

      ax.plot(cylinder[0],cylinder[1],z1,'k')
      ax.plot(cylinder[0],cylinder[1],z2,'k')
      ax.plot(verts[0],verts[1],z1,'k')
      ax.plot(verts[0],verts[1],z2,'k')
      for i in range(N//4):
      ax.plot(edgesin[0,0+2*i:2+2*i],edgesin[1,0+2*i:2+2*i],z1,'k')
      ax.plot(edgesin[0,0+2*i:2+2*i],edgesin[1,0+2*i:2+2*i],z2,'k')

      for i in range(N//4 + N//8):
      ax.plot(edgesout[0,0+2*i:2+2*i],edgesout[1,0+2*i:2+2*i],z1,'k')
      ax.plot(edgesout[0,0+2*i:2+2*i],edgesout[1,0+2*i:2+2*i],z2,'k')

      for i in range(N//4):

plt.plot(edgesout_remain[0,2*i:2+2*i],edgesout_remain[1,2*i:2+2*i],z1,'k')

plt.plot(edgesout_remain[0,2*i:2+2*i],edgesout_remain[1,2*i:2+2*i],z2,'k')
```

```
      # 3D Edges
      vertices3d = np.zeros([3,2*N])
      for i in range(N):
            vertices3d[0,i] = vertices[0,i]
            vertices3d[1,i] = vertices[1,i]
            vertices3d[0,i+N] = vertices[0,i]
            vertices3d[1,i+N] = vertices[1,i]
            vertices3d[2,i] = z1
            vertices3d[2,i+N] = z2

      # rarrange for edges
      edges3d = np.zeros([3,2*N])
      j = 1
      for i in range(2*N):
            if i%2==0:
                  edges3d[:,i] = vertices3d[:,i//2]
            else:
                  edges3d[:,i] = vertices3d[:,i+N-j]
                  j+=1

      for i in range(2*N):

plt.plot(edges3d[0,2*i:2+2*i],edges3d[1,2*i:2+2*i],edges3d[2,2*i:2+2*i],'k'
)

      ax.plot(arcpoints1[0],arcpoints1[1],arcpoints1[2],'g.',label="Arc
Midpoints")
      ax.plot(arcpoints2[0],arcpoints2[1],arcpoints2[2],'g.')
      ax.plot(boundarycircle[0],boundarycircle[1],z1,'k')
      ax.plot(boundarycircle[0],boundarycircle[1],z2,'k')
      ax.plot(x,y,z1,'r.',label=f"Vertices at z={z1:.2e}")
      ax.plot(x,y,z2,'b.',label=f"Vertices at z={z2:.2e}")
      ax.grid()

      ax.legend(fontsize=10)
      plt.show()
```

blockMeshDict Mesh Generation (A)

```
# Script for OF2 : blockmeshdict for Mesh A

mesh = "A"

# creates and writes blockMeshDict for Mesh A


#------------------------------#

import numpy as np
from blockmeshvertices import *

#------------------------------#

def blockMeshDict(intro1,intro2,verts,blocks,edges,faces):
      f = open("OF2\code\\blockMesh\\A\\blockMeshDict", "w")

      f.write(intro1+intro2+verts+blocks+edges+faces)

      f.close()


if __name__ == "__main__":
      # Parameters for Mesh A
      D = 1 # diameter of cylinder
      Lf = 4
      Lw = 8
      R = 1.0
      H = 4

      N = 32 # NOTE: Treat this as a CONST variable
      # DO NOT CHANGE; this blockMeshDict creation does not adapt to higher
number of vertices
      # Due to time constraints with this assignment we stick with N=32
vertices for our simulations
      # However, resolution can be played with in 'blocks' section!

      inner,outer = arcvertices(D,R)
      top,bottom,inlet,outlet = walls(Lf,Lw,H,outer)
      vertices = blockmesh_2D(inner,outer,inlet,top,outlet,bottom)
```

```
      intro1 = f"""
// Mesh {mesh} Generation
// Parameters:
// D (diameter) = {D:.3e}
// Lf (fore)  =  {Lf:.3e}
// Lw (wake)  =  {Lw:.3e}
// R  (outer) =  {R:.3e}
// H  (top/bottom) = {H:.3e}
      """

      intro2 = """
FoamFile
{
      version  2.0;
      format   ascii;
      class dictionary;
      object   blockMeshDict;
}

convertToMeters 1.0;
      """

      verts = """
vertices
(
"""
      z = -5e-02
      for i in range(N):
      vertex = f"       ({vertices[0,i]:.13e} {vertices[1,i]:4e} {z:.13e})
// {i}\n"
      verts +=vertex

      for i in range(N):
      vertex = f"       ({vertices[0,i]:.13e} {vertices[1,i]:4e} {-z:.13e})
// {i+N}\n"
      verts +=vertex

      verts = verts + ");"

      blocks = """\n
blocks
(
```

```
"""

      resX = 10
      resY = 20
      resZ = 1
      gradeX = 2
      gradeY = 1
      gradeZ = 1

      for i in range(N - (N//4+N//8)):
      if i < N//4-1:
            block = f"\
            // block {i}\n\
            hex ({i} {i+N//4} {i+N//4+1} {i+1} {i+N} {i+N//4+N}
{i+N//4+N+1} {N+i+1}) ({resX} {resY} {resZ}) simpleGrading ({gradeX:.4e}
{gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//4 - 1:
            block = f"\
            // block {i}\n\
            hex ({i} {i+N//4} {N//4} {i-(N//4-1)} {i+N} {i+N//4+N} {i+N+1}
{N}) ({resX} {resY} {resZ}) simpleGrading ({gradeX:.4e} {gradeY:.4e}
{gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//4:
            block = f"\
            // block {i}\n\
            hex ({i} {i+N//4} {i+N//4+1} {i+1} {i+N} {i+N//4+N}
{i+N+N//4+1} {N+1+i}) ({resX+20} {resY} {resZ}) simpleGrading
({2*gradeX:.4e} {gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//4+1:
            block = f"\
            // block {i}\n\
            hex ({i} {i+N//4} {i+N//4+1} {i+N//4+2} {i+N} {i+N//4+N}
{i+N//4+N+1} {i+N//4+N+2}) ({resX+20} {resY+10} {resZ}) simpleGrading
({2*gradeX:.4e} {4*gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if N//4+1<i<N//4+N//8:
            block = f"\
            // block {i}\n\
            hex ({i} {i-1} {i+N//4+1} {i+N//4+2} {i+N} {i+N-1} {i+N+N//4+1}
{i+N//4+N+2}) ({resX+10} {resY+10} {resZ}) simpleGrading ({0.5*gradeX:.4e}
```

```
{4*gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//4+N//8:
            block = f"\
            // block {i}\n\
            hex ({i+N//4+3} {i-1} {i+N//4+1} {i+N//4+2} {i+N//4+3+N}
{i+N-1} {i+N//4+3+N-2} {i+N//4+3+N-1}) ({resX+5} {resY+10} {resZ})
simpleGrading ({0.125*gradeX:.4e} {4*gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if N//4+N//8 < i <= N//4+N//8+2:
            block = f"\
            // block {i}\n\
            hex ({i+N//4+3} {i-1} {i-2} {i+N//4+2} {i+N//4+3+N} {i+N-1}
{i+N-2} {i+N//4+3+N-1}) ({resX+5} {resY} {resZ}) simpleGrading
({0.125*gradeX:.4e} {gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//4+N//8 + 3:
            block = f"\
            // block {i}\n\
            hex ({i+N//4+3} {i+N//4+N//8} {i-2} {i+N//4+2} {i+N//4+3+N}
{i+N//4+N//8+N} {i+N-2} {i+N//4+2+N}) ({resX+5} {resY+10} {resZ})
simpleGrading ({0.125*gradeX:.4e} {0.25*gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if N//2 <= i <= N//2 + 1:
            block = f"\
            // block {i}\n\
            hex ({i+N//4+3} {i+N//4+N//8} {i-2} {i-3} {i+N//4+3+N}
{i+N//4+N//8+N} {i+N-2} {i+N-3}) ({resX+10} {resY+10} {resZ}) simpleGrading
({0.5*gradeX:.4e} {0.25*gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//2 + 2:
            block = f"\
            // block {i}\n\
            hex ({i+N//4+N//8-1} {i+N//4+N//8} {i+N//4+N//8+1} {i-3}
{i+N//4+N//8+N-1} {i+N//4+N//8+N} {i+N//4+N//8+N+1} {i+N-3}) ({resX+20}
{resY+10} {resZ}) simpleGrading ({2*gradeX:.4e} {0.25*gradeY:.4e}
{gradeZ:.1e})\n\n"
            blocks+=block
      if i == N - (N//4+N//8) - 1:
            block = f"\
            // block {i}\n\
            hex ({i-N//8} {i+N//4+N//8} {i-N//8+1} {i-N//8-N//4+1}
{i+N-N//8} {i+N + N//8 + N//4} {i+N-N//8+1} {i+N-N//8-N//4+1}) ({resX+20}
```

```
{resY} {resZ}) simpleGrading ({2*gradeX:.4e} {gradeY:.4e}
{gradeZ:.1e})\n\n"
            blocks+=block

      blocks = blocks + ");\n"

      # NOTE: Arcs and Faces from example given shall remain the same given
D,R = 1 and N=32

      midpoints1 = arcmidpoints(z,vertices,D,R)
      midpoints2 = arcmidpoints(-z,vertices,D,R)
      [a,b]=np.shape(midpoints1)

      arcindex1 = np.zeros([2,b])
      arcindex2 = np.zeros([2,b])
      for i in range(b):
      arcindex1[0,i] = i
      arcindex2[0,i] = i+N
      if i != b-1:
            arcindex1[1,i] = i+1
            arcindex2[1,i] = i+1+N
      else:
            arcindex1[1,i] = b/2
            arcindex2[1,i] = b/2+N

      arcindex1[1,N//4-1] = arcindex1[0,0]
      arcindex2[1,N//4-1] = arcindex2[0,0]

      arcs = """
edges
(
"""
      for i in range(b):
      arcs += f"arc {int(arcindex1[0,i])} {int(arcindex1[1,i])} (
{midpoints1[0,i]:.5e} {midpoints1[1,i]:.5e} {midpoints1[2,i]:.5e})\n"

      for i in range(b):
      arcs += f"arc {int(arcindex2[0,i])} {int(arcindex2[1,i])} (
{midpoints2[0,i]:.5e} {midpoints2[1,i]:.5e} {midpoints2[2,i]:.5e})\n"

      arcs = arcs +");\n"

      faces = """
```

```
boundary
(

  inlet
  {
      type patch;
      faces
      (
      (22 54 55 23)
      (23 55 56 24)
      (24 56 57 25)
      (25 57 58 26)
      );
  }

  outlet
  {
      type patch;
      faces
      (
      (18 50 49 17)
      (17 49 48 16)
      (16 48 63 31)
      (31 63 62 30)
      );
  }

  cylinder
  {
      type wall;
      faces
      (
      (0 32 33 1)
      (1 33 34 2)
      (2 34 35 3)
      (3 35 36 4)
      (4 36 37 5)
      (5 37 38 6)
      (6 38 39 7)
      (7 39 32 0)
      );
  }
```

```
  top
  {
      type symmetryPlane;
      faces
      (
      (22 54 53 21)
      (21 53 52 20)
      (20 52 51 19)
      (19 51 50 18)
      );
  }

  bottom
  {
      type symmetryPlane;
      faces
      (
      (26 58 59 27)
      (27 59 60 28)
      (28 60 61 29)
      (29 61 62 30)
      );
  }

);
      """


      blockMeshDict(intro1,intro2,verts,blocks,arcs,faces)
```

blockMeshDict Mesh Generation (B)

```
# Script for OF2 : blockmeshdict for Mesh B

mesh = "B"

# creates and writes blockMeshDict for Mesh B


#------------------------------#

import numpy as np
from blockmeshvertices import *

#------------------------------#

def blockMeshDict(intro1,intro2,verts,blocks,edges,faces):
      f = open("OF2\code\\blockMesh\\B\\blockMeshDict", "w")

      f.write(intro1+intro2+verts+blocks+edges+faces)

      f.close()


if __name__ == "__main__":
      # Parameters for Mesh B (INPUTS)
      D = 1 # diameter of cylinder
      Lf = 5
      Lw = 20
      R = 1
      H = 4

      N = 32 # NOTE: Treat this as a CONST variable
      # DO NOT CHANGE; this blockMeshDict creation does not adapt to higher
number of vertices
      # Due to time constraints with this assignment we stick with N=32
vertices for our simulations
      # However, resolution can be played with in 'blocks' section!

      inner,outer = arcvertices(D,R)
      top,bottom,inlet,outlet = walls(Lf,Lw,H,outer)
      vertices = blockmesh_2D(inner,outer,inlet,top,outlet,bottom)
```

```
      intro1 = f"""
// Mesh {mesh} Generation
// Parameters:
// D (diameter) = {D:.3e}
// Lf (fore)  =  {Lf:.3e}
// Lw (wake)  =  {Lw:.3e}
// R  (outer) =  {R:.3e}
// H  (top/bottom) = {H:.3e}
      """

      intro2 = """
FoamFile
{
      version  2.0;
      format   ascii;
      class dictionary;
      object   blockMeshDict;
}

convertToMeters 1.0;
      """

      verts = """
vertices
(
"""
      z = -5e-02
      for i in range(N):
      vertex = f"       ({vertices[0,i]:.13e} {vertices[1,i]:4e} {z:.13e})
// {i}\n"
      verts +=vertex

      for i in range(N):
      vertex = f"       ({vertices[0,i]:.13e} {vertices[1,i]:4e} {-z:.13e})
// {i+N}\n"
      verts +=vertex

      verts = verts + ");"

      blocks = """\n
blocks
(
```

```
"""

      resX = 10
      resY = 20
      resZ = 1
      gradeX = 2
      gradeY = 1
      gradeZ = 1

      for i in range(N - (N//4+N//8)):
      if i < N//4-1:
            block = f"\
            // block {i}\n\
            hex ({i} {i+N//4} {i+N//4+1} {i+1} {i+N} {i+N//4+N}
{i+N//4+N+1} {N+i+1}) ({resX} {resY} {resZ}) simpleGrading ({gradeX:.4e}
{gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//4 - 1:
            block = f"\
            // block {i}\n\
            hex ({i} {i+N//4} {N//4} {i-(N//4-1)} {i+N} {i+N//4+N} {i+N+1}
{N}) ({resX} {resY} {resZ}) simpleGrading ({gradeX:.4e} {gradeY:.4e}
{gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//4:
            block = f"\
            // block {i}\n\
            hex ({i} {i+N//4} {i+N//4+1} {i+1} {i+N} {i+N//4+N}
{i+N+N//4+1} {N+1+i}) ({resX+20} {resY} {resZ}) simpleGrading
({2*gradeX:.4e} {gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//4+1:
            block = f"\
            // block {i}\n\
            hex ({i} {i+N//4} {i+N//4+1} {i+N//4+2} {i+N} {i+N//4+N}
{i+N//4+N+1} {i+N//4+N+2}) ({resX+20} {resY+10} {resZ}) simpleGrading
({2*gradeX:.4e} {4*gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if N//4+1<i<N//4+N//8:
            block = f"\
            // block {i}\n\
            hex ({i} {i-1} {i+N//4+1} {i+N//4+2} {i+N} {i+N-1} {i+N+N//4+1}
{i+N//4+N+2}) ({resX+10} {resY+10} {resZ}) simpleGrading ({0.5*gradeX:.4e}
```

```
{4*gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//4+N//8:
            block = f"\
            // block {i}\n\
            hex ({i+N//4+3} {i-1} {i+N//4+1} {i+N//4+2} {i+N//4+3+N}
{i+N-1} {i+N//4+3+N-2} {i+N//4+3+N-1}) ({resX+5} {resY+10} {resZ})
simpleGrading ({0.125*gradeX:.4e} {4*gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if N//4+N//8 < i <= N//4+N//8+2:
            block = f"\
            // block {i}\n\
            hex ({i+N//4+3} {i-1} {i-2} {i+N//4+2} {i+N//4+3+N} {i+N-1}
{i+N-2} {i+N//4+3+N-1}) ({resX+5} {resY} {resZ}) simpleGrading
({0.125*gradeX:.4e} {gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//4+N//8 + 3:
            block = f"\
            // block {i}\n\
            hex ({i+N//4+3} {i+N//4+N//8} {i-2} {i+N//4+2} {i+N//4+3+N}
{i+N//4+N//8+N} {i+N-2} {i+N//4+2+N}) ({resX+5} {resY+10} {resZ})
simpleGrading ({0.125*gradeX:.4e} {0.25*gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if N//2 <= i <= N//2 + 1:
            block = f"\
            // block {i}\n\
            hex ({i+N//4+3} {i+N//4+N//8} {i-2} {i-3} {i+N//4+3+N}
{i+N//4+N//8+N} {i+N-2} {i+N-3}) ({resX+10} {resY+10} {resZ}) simpleGrading
({0.5*gradeX:.4e} {0.25*gradeY:.4e} {gradeZ:.1e})\n\n"
            blocks+=block
      if i == N//2 + 2:
            block = f"\
            // block {i}\n\
            hex ({i+N//4+N//8-1} {i+N//4+N//8} {i+N//4+N//8+1} {i-3}
{i+N//4+N//8+N-1} {i+N//4+N//8+N} {i+N//4+N//8+N+1} {i+N-3}) ({resX+20}
{resY+10} {resZ}) simpleGrading ({2*gradeX:.4e} {0.25*gradeY:.4e}
{gradeZ:.1e})\n\n"
            blocks+=block
      if i == N - (N//4+N//8) - 1:
            block = f"\
            // block {i}\n\
            hex ({i-N//8} {i+N//4+N//8} {i-N//8+1} {i-N//8-N//4+1}
{i+N-N//8} {i+N + N//8 + N//4} {i+N-N//8+1} {i+N-N//8-N//4+1}) ({resX+20}
```

```
{resY} {resZ}) simpleGrading ({2*gradeX:.4e} {gradeY:.4e}
{gradeZ:.1e})\n\n"
            blocks+=block

      blocks = blocks + ");\n"

      # NOTE: Arcs and Faces from example given shall remain the same given
D,R = 1 and N=32

      midpoints1 = arcmidpoints(z,vertices,D,R)
      midpoints2 = arcmidpoints(-z,vertices,D,R)
      [a,b]=np.shape(midpoints1)

      arcindex1 = np.zeros([2,b])
      arcindex2 = np.zeros([2,b])
      for i in range(b):
      arcindex1[0,i] = i
      arcindex2[0,i] = i+N
      if i != b-1:
            arcindex1[1,i] = i+1
            arcindex2[1,i] = i+1+N
      else:
            arcindex1[1,i] = b/2
            arcindex2[1,i] = b/2+N

      arcindex1[1,N//4-1] = arcindex1[0,0]
      arcindex2[1,N//4-1] = arcindex2[0,0]

      arcs = """
edges
(
"""
      for i in range(b):
      arcs += f"arc {int(arcindex1[0,i])} {int(arcindex1[1,i])} (
{midpoints1[0,i]:.5e} {midpoints1[1,i]:.5e} {midpoints1[2,i]:.5e})\n"

      for i in range(b):
      arcs += f"arc {int(arcindex2[0,i])} {int(arcindex2[1,i])} (
{midpoints2[0,i]:.5e} {midpoints2[1,i]:.5e} {midpoints2[2,i]:.5e})\n"

      arcs = arcs +");\n"

      faces = """
```

```
boundary
(

  inlet
  {
      type patch;
      faces
      (
      (22 54 55 23)
      (23 55 56 24)
      (24 56 57 25)
      (25 57 58 26)
      );
  }

  outlet
  {
      type patch;
      faces
      (
      (18 50 49 17)
      (17 49 48 16)
      (16 48 63 31)
      (31 63 62 30)
      );
  }

  cylinder
  {
      type wall;
      faces
      (
      (0 32 33 1)
      (1 33 34 2)
      (2 34 35 3)
      (3 35 36 4)
      (4 36 37 5)
      (5 37 38 6)
      (6 38 39 7)
      (7 39 32 0)
      );
  }
```

```
  top
  {
      type symmetryPlane;
      faces
      (
      (22 54 53 21)
      (21 53 52 20)
      (20 52 51 19)
      (19 51 50 18)
      );
  }

  bottom
  {
      type symmetryPlane;
      faces
      (
      (26 58 59 27)
      (27 59 60 28)
      (28 60 61 29)
      (29 61 62 30)
      );
  }

);
      """


      blockMeshDict(intro1,intro2,verts,blocks,arcs,faces)
```